

# MODULE 2

## STATISTICAL MASTERY FOR MACHINE LEARNING AND AI SUCCESS

MLFP — ML Foundations for Professionals — Terrene Open Academy

*"How confident are you in that number?"*

# WHAT YOU WILL LEARN

---

## BY THE END OF MODULE 2

- **Reason about probability**, conditional probability, and Bayes' theorem
- **Estimate parameters** using MLE and bootstrap methods
- **Design and analyse A/B tests** with proper hypothesis testing
- **Build and interpret** linear and logistic regression models
- **Perform ANOVA** for multi-group comparison
- **Implement CUPED** for variance reduction in experiments
- **Engineer features** and manage them in a feature store

## THREE DEPTH LAYERS

**FOUNDATIONS** — Everyone follows. Intuition, analogies, real-world cases.

**THEORY** — Formulas, derivations, KaTeX equations. The mathematical backbone.

**ADVANCED** — Paper references, edge cases, expert insights.

M2 is formula-heavy by design. The green slides always explain the intuition before the blue slides show the math.

## YOUR JOURNEY: 8 LESSONS

---

| #   | Lesson                                  | You Will Be Able To...                             |
|-----|-----------------------------------------|----------------------------------------------------|
| 2.1 | Probability & Bayesian Thinking         | Apply Bayes' theorem to real-world scenarios       |
| 2.2 | Parameter Estimation & Inference        | Compute confidence intervals, implement MLE        |
| 2.3 | Bootstrapping & Hypothesis Testing      | Bootstrap CIs, formulate and test hypotheses       |
| 2.4 | A/B Testing & Experiment Design         | Design experiments, detect SRM, run power analysis |
| 2.5 | Linear Regression                       | Build, interpret, and evaluate regression models   |
| 2.6 | Logistic Regression & Classification    | Predict binary outcomes, interpret odds ratios     |
| 2.7 | CUPED & Causal Inference                | Reduce experiment variance, apply DiD              |
| 2.8 | Capstone — Statistical Analysis Project | Execute end-to-end statistical analysis            |

# KAILASH ENGINES YOU WILL MEET

---

## EXPERIMENTTRACKER

Design A/B tests, track experiment metadata, analyse results with statistical rigour.

## TRAININGPIPELINE

Fit regression and classification models with built-in cross-validation.

## FEATUREENGINEER

Generate interaction terms, polynomial features, temporal features automatically.

## MODELVISUALIZER

Residual plots, coefficient charts, ROC curves — Polars-native.

## FEATURESTORE

Store and retrieve features with point-in-time correctness and data lineage.

**Pattern:** Learn the statistics manually first, then see how the engines automate it. Understanding both means you can debug when results surprise you.

## LESSON 2.1: PROBABILITY AND BAYESIAN THINKING

---

### LEARNING OBJECTIVES

- Construct truth tables for probability problems
- Apply Bayes' theorem to real-world scenarios (medical tests, A/B test results)
- Identify and correct for sampling bias
- Choose appropriate probability distributions for different data types

**Prerequisites:** M1 complete (Python, data manipulation with Polars).

## COVID ART TEST: CAN YOU TRUST THE RESULT?

### A SINGAPORE STORY

- You take a COVID ART test and it reads **positive**
- The test has 95% sensitivity (catches 95% of true positives)
- The test has 97% specificity (correctly clears 97% of negatives)
- Only 1% of Singapore's population is infected right now

#### **Intuition says ~95%.**

The math says ~24%. Most people who test positive in a low-prevalence population do NOT have the disease. This is Bayes' theorem in action.

**Why this matters:** Every ML model is a test. Understanding false positive rates, base rates, and conditional probability is essential for interpreting model outputs.

**Question: What is the probability you actually have COVID?**

# PROBABILITY FUNDAMENTALS

## CORE RULES

- $P(A)$  ranges from 0 (impossible) to 1 (certain)
- $P(A) + P(A') = 1$  — something either happens or it does not
- **Independent events:** outcome of one does not affect the other (coin flips)
- **Dependent events:** outcome of one changes the probability of the other (drawing cards without replacement)

**Decision steps:** (1) Does order matter? (2) Does one event affect the other? If yes to #2, events are dependent.

## TRUTH TABLE: RAIN AND UMBRELLA

|             | Rain | No Rain | Total |
|-------------|------|---------|-------|
| Umbrella    | 0.27 | 0.13    | 0.40  |
| No Umbrella | 0.03 | 0.57    | 0.60  |
| Total       | 0.30 | 0.70    | 1.00  |

Joint probabilities sum to 1. Marginals are the row/column totals.  $P(\text{Rain} \mid \text{Umbrella}) = 0.27 / 0.40 = 0.675$ .

# CONDITIONAL PROBABILITY

## WHAT CHANGES WHEN YOU KNOW SOMETHING?

- **Conditional probability** answers: "Given that B happened, how likely is A?"
- Notation:  $P(A|B)$  — read as "probability of A given B"
- The vertical bar | means "given that we know"

$$P(A, B) = P(A) \times P(B|A)$$

Joint probability = marginal times conditional

## SUPERMARKET EXAMPLE

- 60% of shoppers buy vegetables
- Of those who buy vegetables, 30% also buy olive oil
- $P(\text{Veg}) = 0.60$
- $P(\text{Oil}|\text{Veg}) = 0.30$
- $P(\text{Veg and Oil}) = 0.60 \times 0.30 = 0.18$

**Key insight:** Conditional probability shrinks the universe. Instead of asking about all shoppers, you ask only about the vegetable buyers.

# BAYES' THEOREM

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Posterior = (Likelihood × Prior) / Evidence

## COMPONENTS

| Term     | Name       | Meaning                                  |
|----------|------------|------------------------------------------|
| $P(A B)$ | Posterior  | Updated belief after seeing evidence     |
| $P(B A)$ | Likelihood | How likely is the evidence if A is true? |
| $P(A)$   | Prior      | What you believed before seeing evidence |
| $P(B)$   | Evidence   | Total probability of the evidence        |

## TOTAL PROBABILITY (DENOMINATOR)

$$P(B) = P(B|A)P(A) + P(B|A')P(A')$$

**Intuition:** The denominator sums over all ways B could happen — when A is true and when A is false. It is a normalising constant.

## BAYES IN ACTION: COVID ART TEST

### GIVEN

- $P(\text{COVID}) = 0.01$  (prevalence)
- $P(\text{Pos}|\text{COVID}) = 0.95$  (sensitivity)
- $P(\text{Pos}|\text{No COVID}) = 0.03$  (1 – specificity)

$$\begin{aligned} P(\text{COVID}|\text{Pos}) &= \frac{0.95 \times 0.01}{0.95 \times 0.01 + 0.03 \times 0.99} \\ &= \frac{0.0095}{0.0392} = 0.242 \end{aligned}$$

### PER 10,000 PEOPLE

|          | Test + | Test – | Total  |
|----------|--------|--------|--------|
| COVID    | 95     | 5      | 100    |
| No COVID | 297    | 9,603  | 9,900  |
| Total    | 392    | 9,608  | 10,000 |

**Base rate fallacy:** 297 false positives swamp 95 true positives because the disease is rare. In ML, this is why accuracy is misleading on imbalanced datasets.

## EXPECTED VALUE AND KEY DISTRIBUTIONS

### EXPECTED VALUE

$$E[X] = \sum_i p_i x_i$$

Weighted average: each outcome times its probability

**Example:** A lottery ticket costs \$2. You win \$100 with probability 0.01, \$0 otherwise.  $E[X] = 0.01 \times 100 + 0.99 \times 0 = 1.00$ . Expected value (\$1) is less than cost (\$2). Bad bet.

### THE STATISTICS THEME PARK

| Distribution       | Use When              | Example                   |
|--------------------|-----------------------|---------------------------|
| <b>Normal</b>      | Continuous, symmetric | HDB prices, heights       |
| <b>Poisson</b>     | Count of rare events  | MRT breakdowns/month      |
| <b>Exponential</b> | Time between events   | Time between bus arrivals |
| <b>Beta</b>        | Probabilities (0-1)   | Conversion rates          |
| <b>Uniform</b>     | Equal likelihood      | Random number generator   |

# CONJUGATE PRIORS: ELEGANT BAYESIAN UPDATES

## WHAT IS A CONJUGATE PRIOR?

- A prior distribution is **conjugate** to a likelihood if the posterior has the same family as the prior
- **Beta-Binomial:** Prior =  $\text{Beta}(\alpha, \beta)$ , observe  $k$  successes in  $n$  trials, posterior =  $\text{Beta}(\alpha + k, \beta + n - k)$
- **Normal-Normal:** Prior =  $N(\mu_0, \sigma_0^2)$ , observe data with known variance, posterior is still Normal

## HDB PRICE EXAMPLE

- Prior belief: mean HDB resale price is Beta distributed around \$500K (weak prior)
- Observe 50 transactions
- Posterior shifts toward the observed mean, weighted by how much data we have vs how strong our prior was

**Why it matters:** Conjugate priors give closed-form posteriors — no simulation needed. When priors are not conjugate, you need MCMC (Markov Chain Monte Carlo) sampling.

## SAMPLING BIAS: THE FRIENDSHIP PARADOX

### YOUR FRIENDS HAVE MORE FRIENDS THAN YOU

- On average, your friends have more friends than you do
- This is not pessimism — it is mathematics
- People with many friends appear in more friend lists, so they are oversampled
- Same bias appears everywhere: popular restaurants get more reviews, viral tweets get more impressions

**ML implication:** If you sample users by their activity, you oversample power users. Your model learns from the most active 5%, not the typical user.

**Fix:** Always ask: "How was this data collected? Who is overrepresented? Who is missing?" Random sampling is the gold standard, but rarely achieved in practice.

# KAILASH BRIDGE: PROBABILITY IN PRACTICE

## EXPERIMENTTRACKER — BAYESIAN A/B TESTING

**Manual:** Compute Bayes' theorem, build truth tables, choose priors



**Engine:** ExperimentTracker computes posterior probabilities for experiment variants automatically

```
from kailash_ml import ExperimentTracker

tracker = ExperimentTracker()
tracker.create_experiment(
    name="hdb_price_bayesian",
    hypothesis="Bayesian estimation gives tighter CI than frequentist",
    metrics=["posterior_mean", "credible_interval_width"]
)
# Log prior, likelihood, and posterior for each iteration
tracker.log_metric("posterior_mean", posterior.mean())
tracker.log_metric("ci_width", ci_upper - ci_lower)
```

**Exercise preview:** Bayesian estimation on HDB prices — compute posterior distribution of mean price given prior beliefs and observed data. Visualise prior, likelihood, and posterior.

## LESSON 2.2: PARAMETER ESTIMATION AND INFERENCE

---

### LEARNING OBJECTIVES

- Distinguish population parameters from sample statistics
- Compute confidence intervals and explain their meaning correctly
- Implement MLE for a simple distribution (e.g., Normal parameters)
- Explain when MLE fails and MAP helps

**Prerequisites:** Lesson 2.1 (probability, distributions, Bayes' theorem).

# HOW MUCH IS A 4-ROOM HDB FLAT WORTH?

## THE ESTIMATION PROBLEM

- There are ~350,000 4-room HDB flats in Singapore
- You cannot appraise every single one — that is the **population**
- You observe 200 recent resale transactions — that is your **sample**
- From the sample mean and spread, you estimate the population mean and spread

### Population vs Sample

| Concept | Population       | Sample            |
|---------|------------------|-------------------|
| Mean    | $\mu$ (mu)       | $\bar{x}$ (x-bar) |
| Std Dev | $\sigma$ (sigma) | $s$               |
| Size    | $N$              | $n$               |

Greek letters = population (unknowns). Latin letters = sample (computed from data).

# POPULATION VS SAMPLE VARIANCE

## POPULATION VARIANCE

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

Divide by N when you have the entire population

## SAMPLE VARIANCE (BESSEL'S CORRECTION)

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Divide by n-1 to correct for underestimation

## WHY N-1?

- **Degrees of freedom:** once you know the sample mean, only  $n - 1$  values are free to vary (the last one is determined)
- Dividing by  $n$  systematically **underestimates** the true variance
- $n - 1$  produces an **unbiased** estimate of  $\sigma^2$

**Analogy:** If 5 people must split a \$100 bill evenly, and you know 4 of their shares, you can calculate the 5th. Only 4 are "free." The sample mean constrains one value.

# SAMPLING DISTRIBUTIONS AND THE CENTRAL LIMIT THEOREM

## WHAT IF YOU TOOK MANY SAMPLES?

- Take 1,000 random samples of 200 HDB transactions each
- Compute the mean of each sample
- Plot those 1,000 means — you get a **sampling distribution**
- The spread of the sampling distribution tells you how much your estimate could vary

## CENTRAL LIMIT THEOREM (CLT)

**No matter what shape the original data has**, the distribution of sample means approaches a Normal distribution as sample size grows.

- **Mean of means** = population mean  $\mu$
- **Standard error** =  $\sigma / \sqrt{n}$
- Larger  $n \rightarrow$  narrower sampling distribution  $\rightarrow$  more precise estimate

# CONFIDENCE INTERVALS: WHAT THEY REALLY MEAN

## CORRECT INTERPRETATION

**A 95% confidence interval means:** if we repeated this experiment 100 times, about 95 of those intervals would contain the true population parameter.

**WRONG:** "There is a 95% chance the parameter is in this interval." The parameter is fixed; the interval is random.

## COMPUTING A CI

$$\bar{x} \pm z_{\alpha/2} \cdot \frac{s}{n}$$

For large n:  $z = 1.96$  for 95% CI

- $\bar{x}$  = sample mean
- $z_{\alpha/2}$  = critical value (1.645 for 90%, 1.96 for 95%, 2.576 for 99%)
- $s/\sqrt{n}$  = standard error
- Wider interval = more confident, less precise

# MAXIMUM LIKELIHOOD ESTIMATION (MLE)

## THE BIG IDEA

- You observe data  $x_1, x_2, \dots, x_n$
- You assume a model (e.g., Normal distribution)
- **MLE** finds the parameters  $\theta$  that make the observed data most probable

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \prod_{i=1}^n P(x_i | \theta)$$

## LOG-LIKELIHOOD (PRACTICAL FORM)

$$\ell(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Products become sums in log space — numerically stable

**Why log?** Multiplying many small probabilities causes numerical underflow. Summing log-probabilities is stable and equivalent (log is monotonic).

## WHEN MLE FAILS: SMALL SAMPLES AND MAP

### MLE FAILURE MODES

- **Small n:** MLE overfits to the sample. With 5 data points, the MLE estimate is unreliable.
- **Multimodal likelihood:** Multiple local maxima; optimiser finds the wrong one.
- **Misspecified model:** If data is not Normal but you assume Normal, MLE gives the "best wrong answer."

### MAP: MLE WITH A PRIOR

$$\hat{\theta}_{MAP} = \arg \max_{\theta} \left[ \sum_i \log P(x_i | \theta) + \log P(\theta) \right]$$

MAP = MLE + regularisation from prior beliefs

**Connection:** MAP with a Gaussian prior on  $\theta$  is equivalent to L2 regularisation (Ridge regression). The prior pulls extreme estimates toward reasonable values.

# KAILASH BRIDGE: PARAMETER ESTIMATION

## TRAINING PIPELINE — AUTOMATED MODEL FITTING

**Manual:** Write log-likelihood, optimise, compute CIs



**Engine:** TrainingPipeline fits distributions and models with built-in parameter estimation

```
from kailash_ml import TrainingPipeline, ModelVisualizer

pipeline = TrainingPipeline()
# Fit a model with automatic parameter estimation
pipeline.fit(df, target="resale_price", features=["floor_area_sqm", "remaining_lease"])

# Inspect fitted parameters and their confidence intervals
params = pipeline.get_coefficients()
print(params) # Shows estimate, std_error, t_stat, p_value, ci_lower, ci_upper
```

**Exercise preview:** Fit Normal and Exponential distributions to Singapore economic data. Implement MLE from scratch. Compare MLE vs MAP estimates.

## LESSON 2.3: BOOTSTRAPPING AND HYPOTHESIS TESTING

---

### LEARNING OBJECTIVES

- Implement bootstrap resampling and compute confidence intervals
- Formulate null and alternative hypotheses for business questions
- Compute and interpret p-values correctly
- Apply multiple testing corrections (Bonferroni, BH-FDR)

**Prerequisites:** Lesson 2.2 (sampling distributions, confidence intervals).

## WHEN THEORY IS NOT ENOUGH

### THE PROBLEM

- You want a confidence interval for the **median** HDB resale price
- The formula-based CI from Lesson 2.2 works for the mean, but the median has no simple formula for its standard error
- You have only one sample of 200 transactions
- Getting more samples costs time and money

**Bootstrap idea:** If you cannot get more samples from the population, create "new" samples from the sample you have. Resample *with replacement* thousands of times. Each resample gives you a new estimate of your statistic.

**Key insight:** The bootstrap treats your sample as a stand-in for the population. The variation across resamples estimates the variation you would see if you could repeat the study.

## BOOTSTRAP: THE RESAMPLING ALGORITHM

### STEPS

1. Start with original sample of size  $n$
2. Draw  $n$  observations **with replacement** (bootstrap sample)
3. Compute your statistic on the bootstrap sample (bootstrap replicate)
4. Repeat steps 2-3 many times (e.g.,  $B = 10,000$ )
5. The distribution of replicates is the **bootstrap distribution**

### BOOTSTRAP CI: PERCENTILE METHOD

$$CI_{1-\alpha} = \left[ \hat{\theta}_{\alpha/2}^*, \hat{\theta}_{1-\alpha/2}^* \right]$$

Sort the  $B$  replicates. Take the  $\alpha/2$  and  $1 - \alpha/2$  percentiles.

**Example (95% CI):** Sort 10,000 bootstrap medians. The 250th smallest and 9,750th smallest give you the interval.

**No distributional assumption!** Unlike the z-based CI, bootstrap works for any statistic (median, IQR, correlation, ratio).

# HYPOTHESIS TESTING: MAKING DECISIONS

## FRAMEWORK

- **Null hypothesis ( $H_0$ ):** "No effect" or "no difference"
- **Alternative ( $H_1$ ):** "There IS an effect"
- **p-value:** probability of observing data at least as extreme as what you got, *assuming  $H_0$  is true*

**p-value is NOT:**  $P(H_0 \text{ is true})$ . It is  $P(\text{data this extreme} | H_0)$ . The probability of the data, not the hypothesis.

## DECISION RULES

| $\alpha$ Level | Threshold | Field                    |
|----------------|-----------|--------------------------|
| 0.10           | Lenient   | Exploratory research     |
| 0.05           | Standard  | Social science, business |
| 0.01           | Strict    | Medical, regulatory      |
| 0.0005         | 5-sigma   | Particle physics         |

**Language:** "Reject  $H_0$ " or "Fail to reject  $H_0$ ". Never say "accept  $H_0$ " — absence of evidence is not evidence of absence.

# TEST STATISTICS AND PERMUTATION TESTS

## T-STATISTIC

$$T = \frac{\bar{x} - \mu_0}{s / \sqrt{n}}$$

How many standard errors is the sample mean from the hypothesised value?

- Large  $|T| \rightarrow$  small p-value  $\rightarrow$  reject  $H_0$
- One-tailed: test in one direction only
- Two-tailed: test in both directions

## PERMUTATION TEST (NON-PARAMETRIC)

1. Combine control and treatment groups
2. Randomly shuffle group labels
3. Compute the test statistic on the shuffled data
4. Repeat thousands of times
5. p-value = fraction of shuffled statistics at least as extreme as the observed one

**Advantage:** No distributional assumptions. Works for any test statistic. The gold standard for A/B test analysis.

## MULTIPLE TESTING: THE BONFERRONI TRAP

### THE PROBLEM

- You test 20 metrics in your A/B test at  $\alpha = 0.05$
- Even if the treatment has NO effect, you expect  $20 \times 0.05 = 1$  false positive
- The more tests you run, the more "significant" results you get by chance

### BONFERRONI CORRECTION

$$\alpha_{\text{adjusted}} = \frac{\alpha}{m}$$

$m$  = number of tests. Conservative: controls family-wise error rate.

### BENJAMINI-HOCHBERG (BH-FDR)

- Less conservative than Bonferroni
- Controls the **false discovery rate** (expected proportion of false positives among rejections)
- Sort p-values, compare each to  $\frac{i}{m} \alpha$
- Better for exploratory analysis with many comparisons

**Rule of thumb:** Use Bonferroni for confirmatory tests (few, pre-registered). Use BH-FDR for exploratory tests (many metrics, screening).

## POWER ANALYSIS: HOW MUCH DATA DO YOU NEED?

### FOUR COMPONENTS

- **Effect size ( $\delta$ ):** the minimum difference you want to detect
- **Sample size ( $n$ ):** how many observations per group
- **Significance level ( $\alpha$ ):** false positive rate (usually 0.05)
- **Power ( $1 - \beta$ ):** probability of detecting a real effect (usually 0.80)

Fix any three; solve for the fourth.

### MINIMUM SAMPLE SIZE

$$n = \frac{(z_{\alpha/2} + z_{\beta})^2 \cdot 2\sigma^2}{\delta^2}$$

For a two-sample test with equal group sizes

**Common mistake:** Running an experiment without power analysis, then finding "no significant effect." You may simply have been underpowered — not enough data to detect a real effect.

# KAILASH BRIDGE: BOOTSTRAP AND TESTING

## EXPERIMENTTRACKER — AUTOMATED HYPOTHESIS TESTING

**Manual:** Write bootstrap loop, compute CI, run permutation test



**Engine:** ExperimentTracker runs bootstrap, permutation, and power analysis in one call

```
from kailash_ml import ExperimentTracker

tracker = ExperimentTracker()
experiment = tracker.create_experiment(
    name="conversion_ab_test",
    hypothesis="New checkout flow increases conversion rate",
    metrics=["conversion_rate", "revenue_per_user"]
)
# Built-in bootstrap CI and permutation test
results = tracker.analyze(
    control=control_data, treatment=treatment_data,
    method="permutation", n_bootstrap=10000, alpha=0.05
)
print(results.p_value, results.bootstrap_ci, results.power)
```

**Exercise preview:** A/B test analysis with multiple corrections. Bootstrap CI for conversion rate difference. Permutation test for significance.

## LESSON 2.4: A/B TESTING AND EXPERIMENT DESIGN

---

### LEARNING OBJECTIVES

- Design an A/B test with proper randomisation and power analysis
- Create a data collection plan (what, where, how, frequency)
- Detect Sample Ratio Mismatch (SRM) in experiment data
- Identify common data collection pitfalls and mitigations

**Prerequisites:** Lesson 2.3 (hypothesis testing, p-values, power analysis).

# SINGAPORE HAWKER CENTRE: SHOULD WE RAISE PRICES?

## THE BUSINESS QUESTION

- A hawker centre stall wants to test whether a **\$0.50 price increase** on chicken rice reduces sales volume
- Hypothesis: "The price increase does not significantly reduce daily revenue"
- They run the new price on weekdays, old price on weekends

**Problem:** Weekday vs weekend customers are different populations. This is NOT a valid A/B test — it is a confounded comparison.

## PROPER DESIGN

- **Randomise:** Show the old price to 50% of customers and the new price to 50% (random by receipt number)
- **Power analysis:** How many customers do we need to detect a 5% drop in revenue?
- **Duration:** Run for 4 weeks to capture variation across paydays and weather
- **SRM check:** Verify the 50/50 split is actually 50/50

# DATA COLLECTION FRAMEWORK: WHY, WHAT, WHERE, HOW

---

## WHY

- What hypotheses are being tested?
- What is the value and impact of the answer?
- What performance measures matter?

## WHAT

- Wish list of ideal information
- Budget and time constraints
- Minimum viable dataset

## WHERE

- **Internal:** CRM, POS systems, app analytics, finance systems
- **External:** data.gov.sg, Kaggle, public APIs, purchased datasets

## HOW + FREQUENCY

- **At start:** Review existing data, discover gaps, validate quality
- **Continuous:** Automated real-time or batch pipelines
- Per transaction vs batch collection
- Duration must be sufficient for statistical power

## SAMPLE RATIO MISMATCH (SRM)

### WHAT IS SRM?

- You expected a 50/50 split but observed 52/48
- Is this random variation, or is something systematically wrong?
- SRM invalidates your entire experiment — the groups are no longer comparable

### COMMON CAUSES

- Bot filtering applied unevenly
- Redirect bugs (one variant loads faster)
- Population filtering (login-required vs guest)

### CHI-SQUARED SRM TEST

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

$O_i$  = observed count,  $E_i$  = expected count

**Example:** Expected 5,000 / 5,000. Observed 5,200 / 4,800.

$$\chi^2 = \frac{(5200-5000)^2}{5000} + \frac{(4800-5000)^2}{5000} = \frac{40000}{5000} + \frac{40000}{5000} = 16$$

With 1 degree of freedom,  $\chi^2 = 16$  gives  $p \ll 0.001$ . SRM detected.

## COMMON EXPERIMENT PITFALLS

| Pitfall                       | What Goes Wrong                                                    |
|-------------------------------|--------------------------------------------------------------------|
| <b>No randomisation</b>       | Groups differ systematically; cannot attribute effect to treatment |
| <b>No power analysis</b>      | Experiment too small to detect real effects                        |
| <b>Peeking</b>                | Checking results early inflates false positive rate                |
| <b>Overlapping treatments</b> | Users in multiple experiments; effects interact                    |
| <b>Temporal bias</b>          | Treatment coincides with external events (holidays, news)          |

### Clean DataOps Architecture

- **Data schema:** Define what fields you collect upfront
- **Data transfer:** Automated pipelines, not manual exports
- **Processing logic:** Version-controlled transformations
- **Connectors:** Standard interfaces to internal/external sources

**Data silos** are the #1 barrier. Different departments collect overlapping data in incompatible formats.

# KAILASH BRIDGE: EXPERIMENT DESIGN

## EXPERIMENTTRACKER — END-TO-END EXPERIMENT MANAGEMENT

**Manual:** Randomise, collect, check SRM, analyse, report



**Engine:** ExperimentTracker manages the full experiment lifecycle with built-in SRM detection

```
from kailash_ml import ExperimentTracker

tracker = ExperimentTracker()
experiment = tracker.create_experiment(
    name="hawker_price_test",
    hypothesis="$0.50 increase does not reduce daily revenue",
    variants=["control_price", "new_price"],
    allocation=[0.5, 0.5],
    primary_metric="daily_revenue",
    power=0.80, alpha=0.05, mde=0.05
)
# SRM check runs automatically at analysis time
results = tracker.analyze(experiment_id=experiment.id)
print(results.srm_check) # Pass/Fail with chi-squared p-value
```

**Exercise preview:** Design and analyse a complete A/B experiment. Create data collection plan. Detect SRM. Report results with business recommendation.

## LESSON 2.5: LINEAR REGRESSION

---

### LEARNING OBJECTIVES

- Build and interpret a multivariate linear regression
- Test coefficient significance using t-statistics
- Evaluate model fit using R-squared and F-statistic
- Handle categorical variables with proper dummy encoding
- Introduce non-linearity through interaction and polynomial terms

**Prerequisites:** Lesson 2.2 (MLE, parameter estimation), Lesson 2.3 (hypothesis testing, p-values).

# HOW MUCH IS YOUR HDB FLAT WORTH?

## THE PREDICTION PROBLEM

- You want to predict the resale price of a 4-room HDB flat
- Features: floor area, remaining lease, storey, distance to MRT, town
- **Linear regression** fits a line (or hyperplane) through the data
- Each coefficient tells you: "holding everything else constant, one more unit of this feature changes the price by this much"

### The regression equation:

$$\text{price} = \beta_0 + \beta_1 \cdot \text{area} + \beta_2 \cdot \text{lease} + \beta_3 \cdot \text{storey} + \epsilon$$

$\beta_0$  = base price,  $\beta_1$  = price per sqm,  $\epsilon$  = error (what the model cannot explain)

**Regression is an inference tool first.** We use it to understand relationships, not just to predict. "Does floor area significantly affect price?" is a hypothesis test on  $\beta_1$ .

# ORDINARY LEAST SQUARES (OLS)

## MINIMISE SQUARED RESIDUALS

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \min_{\beta} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2$$

Find coefficients that minimise the sum of squared errors

## CLOSED-FORM SOLUTION (NORMAL EQUATIONS)

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Matrix form: project  $\mathbf{y}$  onto the column space of  $\mathbf{X}$

## WHAT EACH PIECE MEANS

- $y_i$  = actual observed value
- $\hat{y}_i$  = model prediction
- $y_i - \hat{y}_i$  = residual (error)
- $\mathbf{X}$  = matrix of features (each row is one observation)
- $\beta$  = vector of coefficients

**Why squared?** Squaring penalises large errors more than small ones AND ensures a unique minimum (the loss surface is convex).

# TESTING COEFFICIENTS: THE T-STATISTIC

## IS THIS COEFFICIENT REAL?

$$t = \frac{\hat{\beta}}{SE(\hat{\beta})}$$

Signal / Noise: how many standard errors is the coefficient from zero?

- $H_0: \beta = 0$  (this feature has no effect)
- $H_1: \beta \neq 0$  (this feature matters)
- Large  $|t| \rightarrow$  small p-value  $\rightarrow$  reject  $H_0$

## QUICK REFERENCE

| Confidence | $ t $ Cutoff | Stars |
|------------|--------------|-------|
| 90%        | > 1.6        | *     |
| 95%        | > 1.8        | **    |
| 99%        | > 1.97       | ***   |

**Interpretation:** If  $\hat{\beta}_{\text{area}} = 5200$  with  $t = 12.3$  and  $p < 0.001$ , it means: "Each additional square metre is associated with a \$5,200 increase in resale price, and this is highly statistically significant."

# MODEL EVALUATION: R-SQUARED AND F-STATISTIC

## R-SQUARED: VARIANCE EXPLAINED

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$R^2 = 0$ : model explains nothing.  $R^2 = 1$ : perfect fit.

- $SS_{\text{res}}$  = sum of squared residuals (model error)
- $SS_{\text{tot}}$  = total sum of squares (data variance)
- **Adjusted  $R^2$**  penalises for adding useless predictors

## F-STATISTIC: IS THE MODEL USEFUL?

$$F = \frac{SS_{\text{reg}}/k}{SS_{\text{res}}/(n - k - 1)}$$

Tests  $H_0$ : all coefficients are zero (model is useless)

- $k$  = number of predictors
- $SS_{\text{reg}} = SS_{\text{tot}} - SS_{\text{res}}$
- $p < 0.01 \rightarrow$  model is better than predicting the mean

**T-stat tests one coefficient.** F-stat tests the entire model. Both matter.

## CATEGORICAL VARIABLES: DUMMY ENCODING

### THE PROBLEM

- Town is a categorical variable: Ang Mo Kio, Bedok, Clementi, ...
- Regression needs numbers, not text
- **Solution:** Create binary (0/1) columns for each category
- Drop one category as the **base/benchmark**

**Dummy variable trap:** If you include all categories,  $\mathbf{X}^T \mathbf{X}$  is singular (perfect multicollinearity). Always drop one.

### EXAMPLE: SALARY MODEL

$$\text{salary} = \beta_0 + \beta_1 \cdot \text{age} + \beta_2 \cdot \text{age}^2 + \beta_3 \cdot \text{female} + \beta_4 \cdot \text{trans}$$

Base category: male.  $\beta_3$  = salary difference for female vs male, holding age constant.

Note:  $\text{age}^2$  introduces non-linearity — salary peaks at some age then declines. The quadratic term captures this curve.

## BEYOND STRAIGHT LINES: NON-LINEAR TERMS

### POLYNOMIAL TERMS ( $X^2$ )

- Add  $\text{age}^2$  to model a curve: salary increases with age, then levels off
- Add  $\text{area}^2$  if price per sqm changes for very large flats
- Still a "linear" model — linear in the *coefficients*, not in the features

### LOG-LINEAR ( $\text{LN } Y$ )

- $\ln(\text{price}) = \beta_0 + \beta_1 \cdot \text{area}$
- $\beta_1$  now means: "a 1-unit increase in area is associated with a  $\beta_1 \times 100\%$  change in price"

### INTERACTION TERMS ( $X_1 \cdot X_2$ )

- "The effect of area depends on the town"
- $\text{price} = \beta_0 + \beta_1 \cdot \text{area} + \beta_2 \cdot \text{central} + \beta_3 \cdot (\text{area} \times \text{central})$
- $\beta_3$  captures how much MORE each sqm is worth in central vs non-central towns

**Cross-validation preview:** More terms = better training fit but risk of overfitting. Train/test split and k-fold cross-validation (detailed in M3.2) help detect this.

## MULTIVARIATE REGRESSION: CETERIS PARIBUS

### HOLDING EVERYTHING ELSE CONSTANT

- In simple regression,  $\beta_1$  is the raw correlation slope
- In multivariate regression,  $\beta_1$  is the **partial effect** — the effect of  $x_1$  after "stripping out" the effect of all other predictors
- This is what makes regression powerful for causal reasoning (with caveats)

### SIMPSON'S PARADOX WARNING

**Example:** A simple regression shows area has a *negative* coefficient for price (larger flats cost less). But when you add "town" to the model, the coefficient flips to positive. Why? Larger flats are in cheaper towns. Town confounds the area-price relationship.

Multivariate regression "controls for" confounders, but only the ones you include. Omitted variable bias is always a risk.

# KAILASH BRIDGE: LINEAR REGRESSION

## TRAINING PIPELINE + MODEL VISUALIZER — REGRESSION MODELLING

**Manual:** OLS formula, t-stats, R-squared, residual plots



**Engine:** TrainingPipeline fits models; ModelVisualizer creates diagnostic plots

```
from kailash_ml import TrainingPipeline, ModelVisualizer, FeatureEngineer

# Engineer features: lat/lon, interaction terms
fe = FeatureEngineer()
df = fe.add_polynomial(df, columns=["floor_area_sqm"], degree=2)
df = fe.add_interaction(df, col_a="floor_area_sqm", col_b="is_central")

# Fit regression
pipeline = TrainingPipeline()
pipeline.fit(df, target="resale_price", model="linear_regression")

# Evaluate: coefficients, t-stats, R-squared, F-statistic
print(pipeline.summary())

# Visualise residuals
viz = ModelVisualizer()
viz.residual_plot(pipeline)
```

**Exercise preview:** Build HDB price prediction model. Engineer lat/lon features. Interpret coefficients. Test significance. Add interaction terms and compare.

## LESSON 2.6: LOGISTIC REGRESSION AND CLASSIFICATION

---

### LEARNING OBJECTIVES

- Build and interpret a logistic regression model
- Explain the sigmoid function and log-odds transformation
- Compute and interpret odds ratios
- Perform one-way ANOVA and interpret post-hoc tests
- Know when to use ANOVA vs t-test vs regression

**Prerequisites:** Lesson 2.5 (linear regression, coefficients, significance testing).

## WILL THIS EMPLOYEE RESIGN?

---

### THE PROBLEM WITH LINEAR REGRESSION

- Outcome: resigned (1) or stayed (0)
- If you use linear regression, predictions can be **below 0 or above 1**
- A prediction of  $-0.3$  or  $1.5$  for a probability makes no sense
- We need a model that outputs values **between 0 and 1**

**Logistic regression solves this** by wrapping the linear equation in a function that squashes the output into  $[0, 1]$ . That function is the **sigmoid**.

**Common mistake:** Using linear regression for binary outcomes. The predictions are uninterpretable and the error assumptions are violated. Always use logistic regression for yes/no outcomes.

# THE SIGMOID FUNCTION

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Maps any real number  $z$  to  $(0, 1)$

## PROPERTIES

- $\sigma(0) = 0.5$  (decision boundary)
- $\sigma(z) \rightarrow 1$  as  $z \rightarrow +\infty$
- $\sigma(z) \rightarrow 0$  as  $z \rightarrow -\infty$
- S-shaped curve, smooth and differentiable
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$  — elegant derivative

## WHERE $Z$ COMES FROM

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

The same linear equation from linear regression

Logistic regression = linear regression + sigmoid wrapper.  
The linear part computes a score ( $z$ ). The sigmoid converts it to a probability.

**Deep learning preview:** The sigmoid is an *activation function*. In M4 (neural networks), you will use sigmoid, ReLU, and other activations to introduce non-linearity.

# LOG-ODDS AND ODDS RATIOS

## LOG-ODDS TRANSFORMATION

$$\log \frac{P}{1-P} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

The left side maps (0,1) to  $(-\infty, +\infty)$

- **Odds:**  $P/(1-P)$ . If  $P = 0.80$ , odds = 4:1 ("four times more likely to happen than not")
- **Log-odds (logit):**  $\log(P/(1-P))$ . The natural log of the odds
- Coefficients are on the log-odds scale

## INTERPRETING COEFFICIENTS

$$\text{Odds Ratio} = e^{\beta_1}$$

Multiplicative change in odds for a 1-unit increase in  $x_1$

**Example:** If  $\beta_{\text{overtime}} = 1.39$ , then  $e^{1.39} = 4.01$ . Employees who work overtime have **4x the odds** of resigning compared to those who do not.

**Direction check:**  $\beta > 0$  means higher odds.  $\beta < 0$  means lower odds.  $\beta = 0$  means no effect (odds ratio = 1).

## FITTING LOGISTIC REGRESSION: MLE (NOT OLS)

### WHY NOT OLS?

- OLS minimises squared errors — works for continuous outcomes
- For binary outcomes (0/1), squared error is not the right loss
- Logistic regression uses **Maximum Likelihood Estimation**
- Find coefficients that maximise the probability of observing the data

### LOG-LIKELIHOOD FOR BINARY OUTCOME

$$\ell(\beta) = \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

$\hat{p}_i = \sigma(\mathbf{x}_i^T \beta)$  — the predicted probability

**This is cross-entropy loss** — the same loss used in neural networks for classification. You will see it again in M4.

## BEYOND BINARY: MULTICLASS AND EVALUATION PREVIEW

### MULTICLASS LOGISTIC REGRESSION

- **One-vs-Rest (OvR):** Train  $K$  binary classifiers, each separating one class from the rest
- **One-vs-One (OvO):** Train  $\binom{K}{2}$  classifiers for each pair of classes
- **Multinomial:** Single model with softmax activation (generalised sigmoid)

### EVALUATION PREVIEW

| Metric           | What It Measures                         |
|------------------|------------------------------------------|
| Accuracy         | % of correct predictions                 |
| Confusion Matrix | TP, FP, TN, FN breakdown                 |
| Precision        | Of predicted positives, % truly positive |
| Recall           | Of actual positives, % correctly found   |

**Detailed metrics in M3.5.** For now, know that accuracy alone is misleading on imbalanced data (recall the COVID test base rate problem from Lesson 2.1).

## ANOVA: COMPARING THREE OR MORE GROUPS

### WHEN TO USE ANOVA

- T-test compares **2 groups**
- ANOVA generalises to **3+ groups**
- $H_0$ : all group means are equal
- $H_1$ : at least one group mean differs

$$F = \frac{MS_{\text{between}}}{MS_{\text{within}}} = \frac{SS_{\text{between}} / (k - 1)}{SS_{\text{within}} / (N - k)}$$

Between-group variation / Within-group variation

### SINGAPORE HDB EXAMPLE

- Compare resale prices across flat types: 3-room, 4-room, 5-room, executive
- $F = 342.5, p < 0.001 \rightarrow$  prices differ significantly across flat types
- But *which* types differ? ANOVA does not tell you

### POST-HOC TESTS

- **Tukey's HSD**: Compare all pairs, controls family-wise error
- **Bonferroni**: More conservative (from Lesson 2.3)
- **Scheffe**: Most conservative, for complex contrasts

## ANOVA VS T-TEST VS REGRESSION: WHEN TO USE WHAT

| Method                     | Groups | Outcome    | Predictors     | Use When                            |
|----------------------------|--------|------------|----------------|-------------------------------------|
| <b>T-test</b>              | 2      | Continuous | 1 categorical  | Simple comparison                   |
| <b>ANOVA</b>               | 3+     | Continuous | 1+ categorical | Multi-group comparison              |
| <b>Linear Regression</b>   | Any    | Continuous | Mixed          | Continuous + categorical predictors |
| <b>Logistic Regression</b> | Any    | Binary     | Mixed          | Yes/no outcome                      |

**Deep connection:** ANOVA with one factor is equivalent to a linear regression with dummy variables for the factor. The F-statistic is the same. In practice, regression is more flexible because it handles both categorical and continuous predictors simultaneously.

## KAILASH BRIDGE: CLASSIFICATION AND ANOVA

### TRAININGPIPELINE + MODELVISUALIZER — CLASSIFICATION MODELLING

**Manual:** Sigmoid, log-odds, MLE, ANOVA F-test



**Engine:** TrainingPipeline fits logistic models; ModelVisualizer plots ROC curves and confusion matrices

```
from kailash_ml import TrainingPipeline, ModelVisualizer

pipeline = TrainingPipeline()
pipeline.fit(df, target="attrition", model="logistic_regression")

# Odds ratios for each feature
coeffs = pipeline.get_coefficients()
coeffs["odds_ratio"] = coeffs["estimate"].apply(lambda b: math.exp(b))
print(coeffs[["feature", "estimate", "odds_ratio", "p_value"]])

# Confusion matrix and ROC curve
viz = ModelVisualizer()
viz.confusion_matrix(pipeline)
viz.roc_curve(pipeline)
```

**Exercise preview:** Build logistic regression for employee attrition. Interpret odds ratios. Perform ANOVA on HDB prices across flat types. Apply Tukey's HSD.

## LESSON 2.7: CUPED AND CAUSAL INFERENCE

---

### LEARNING OBJECTIVES

- Implement CUPED to reduce A/B test variance by up to 50%
- Detect SRM in experiment data
- Apply Difference-in-Differences (DiD) when randomisation is impossible
- Test the parallel trends assumption

**Prerequisites:** Lesson 2.4 (A/B testing), Lesson 2.5 (linear regression).

## THE NOISY EXPERIMENT PROBLEM

### YOUR A/B TEST RAN FOR 4 WEEKS

- You tested a new checkout flow at a Singapore e-commerce site
- The treatment group showed 2% higher revenue per user
- But the p-value is 0.12 — **not significant**
- Running the experiment longer is expensive

**Problem:** Revenue per user is highly variable. Big spenders and window shoppers are in the same sample. The noise drowns out the signal.

**CUPED solution:** You have pre-experiment data — each user's revenue last month. Adjust the post-experiment metric using the pre-experiment metric. The noise drops, the signal emerges, and now  $p = 0.03$ .

CUPED = **C**ontrolled-experiment **U**sing **P**re-**E**xperiment **D**ata. Introduced by Microsoft (Deng et al., 2013).

# CUPED: THE MATHEMATICS

## VARIANCE REDUCTION FORMULA

$$\text{Var}(Y_{\text{adj}}) = \text{Var}(Y)(1 - \rho^2)$$

$\rho$  = correlation between pre- and post-experiment metrics

- If  $\rho = 0.7$ , variance reduction =  $1 - 0.49 = 51\%$
- If  $\rho = 0.5$ , variance reduction =  $1 - 0.25 = 75\%$  remaining (25% reduction)
- Higher correlation = more variance removed

## CUPED ESTIMATOR

$$Y_{\text{adj}} = Y - \theta \cdot (X_{\text{pre}} - E[X_{\text{pre}}])$$

Subtract out the predictable variation

$$\theta = \frac{\text{Cov}(Y, X_{\text{pre}})}{\text{Var}(X_{\text{pre}})}$$

Optimal  $\theta$  is the OLS regression coefficient of  $Y$  on  $X_{\text{pre}}$

**CUPED IS regression.** It regresses the post-experiment metric on the pre-experiment metric and uses the residual. Same math, different framing.

## WHEN CUPED HELPS MOST

| Scenario                              | $\rho$  | Variance Reduction |
|---------------------------------------|---------|--------------------|
| Revenue: last month vs this month     | 0.7–0.9 | 50–80%             |
| Session count: last week vs this week | 0.6–0.8 | 35–65%             |
| New feature adoption (no history)     | 0.0–0.2 | 0–4%               |

**Rule of thumb:** CUPED works when users behave consistently over time. If past behaviour predicts future behaviour ( $\rho > 0.5$ ), CUPED is worth using.

**CUPED does NOT work for:** new users (no pre-experiment data), completely new features (no historical analog), one-time events (no repeat behaviour).

## DIFFERENCE-IN-DIFFERENCES (DID)

### WHEN RANDOMISATION IS IMPOSSIBLE

- Policy changes, regulatory shifts, natural experiments
- You observe a treatment group and a control group **before and after** the intervention
- DiD estimates the **causal effect** by comparing the changes

$$ATT = (\bar{Y}_{\text{treat,post}} - \bar{Y}_{\text{treat,pre}}) - (\bar{Y}_{\text{ctrl,post}} - \bar{Y}_{\text{ctrl,pre}})$$

ATT = Average Treatment Effect on the Treated

### SINGAPORE COOLING MEASURES

- **Treatment:** HDB resale market (ABSD applied)
- **Control:** Comparable market without the policy change
- Compare price trends before/after the cooling measure
- The "second difference" removes confounders that affect both groups equally

**Critical assumption:** Parallel trends — without the treatment, both groups would have followed the same trajectory. Test this with pre-period data.

## TESTING PARALLEL TRENDS

### PRE-PERIOD VALIDATION

- Plot both groups' outcomes over the pre-treatment period
- If the lines are parallel (same slope, different levels), the assumption holds
- If they diverge before treatment, DiD is unreliable

### PLACEBO TEST

- Pretend the treatment happened at a different (pre-period) time
- Run DiD with the fake treatment date
- If you find a "significant effect" at the fake date, the parallel trends assumption is suspect

### Beyond DiD: Propensity Score Matching

When no natural control group exists, match treated units to similar untreated units based on observed characteristics. Deferred to reference material due to complexity.

**The causal inference ladder:** A/B test (gold standard) → DiD (natural experiments) → Propensity matching (observational) → Instrumental variables (econometrics). Each step weakens causal claims.

# KAILASH BRIDGE: CUPED AND CAUSAL INFERENCE

## EXPERIMENTTRACKER — CUPED AND DID ANALYSIS

**Manual:** Compute theta, adjust metrics, test parallel trends



**Engine:** ExperimentTracker applies CUPED adjustment and runs DiD analysis automatically

```
from kailash_ml import ExperimentTracker
tracker = ExperimentTracker()

# CUPED analysis
results = tracker.analyze(
    control=control_data, treatment=treatment_data,
    pre_experiment_covariate="revenue_last_month",
    method="cuped"
)
print(f"Variance reduction: {results.variance_reduction_pct:.1f}%")
print(f"Adjusted p-value: {results.p_value:.4f}")

# DiD analysis
did_results = tracker.did_analysis(
    treatment=db_treated, control=db_control,
    pre_period="2019-01-01:2020-06-30",
    post_period="2020-07-01:2021-12-31"
)
print(f"ATT: {did_results.att:.2f}, p-value: {did_results.p_value:.4f}")
```

**Exercise preview:** Implement CUPED on experiment data. Compare variance with and without CUPED. DiD analysis on Singapore cooling measures policy.

## LESSON 2.8: CAPSTONE — STATISTICAL ANALYSIS PROJECT

---

### LEARNING OBJECTIVES

- Execute a complete statistical analysis from data to recommendations
- Engineer temporal and interaction features
- Store and retrieve features with point-in-time correctness
- Present statistical findings to a non-technical audience

**Prerequisites:** All of Module 2 (Lessons 2.1–2.7).

# FROM STATISTICS TO DECISION: THE FULL PIPELINE

---

## END-TO-END WORKFLOW

1. **Load** — Import data (from M1)
2. **Describe** — Summary statistics, distributions
3. **Hypothesise** — Formulate testable questions
4. **Test** — Bootstrap, hypothesis tests, ANOVA
5. **Model** — Regression or logistic regression
6. **Interpret** — Coefficients, significance, model fit
7. **Report** — Business recommendations

## PROJECT OPTIONS

**Option A:** Wine Quality Analysis — predict quality from chemical properties. Regression + feature engineering.

**Option B:** Singapore Economic Indicators — model GDP growth from trade, employment, and inflation data.

**Option C:** Experiment Design & Analysis — design, run, and analyse a complete A/B experiment with CUPED.

# FEATURE ENGINEERING: CREATING BETTER INPUTS

## WHAT IS FEATURE ENGINEERING?

- **Features** are the inputs to your model
- **Feature engineering** is creating new, more informative features from raw data
- Good features matter more than complex models

## TYPES OF ENGINEERED FEATURES

- **Temporal:** Day of week, month, quarter, time since last event
- **Interaction:** area × town, age × income
- **Polynomial:**  $x^2$ ,  $x^3$  for non-linear relationships
- **Aggregation:** Rolling means, counts, ratios

### Domain knowledge drives feature engineering.

Knowing that HDB prices depend on remaining lease, proximity to MRT, and school zones is more valuable than any algorithm.

```
from kailash_ml import FeatureEngineer

fe = FeatureEngineer()
df = fe.add_temporal(df, date_col="transaction_date")
# Creates: day_of_week, month, quarter,
# year, is_weekend, days_since_epoch
df = fe.add_interaction(
    df, col_a="floor_area", col_b="storey"
)
# Creates: floor_area_x_storey
```

# FEATURESTORE: PREVENTING DATA LEAKAGE

## THE LEAKAGE PROBLEM

- You train a model to predict January sales using features computed from all of 2024
- But in January, you did not know February-December data!
- **Point-in-time correctness:** features must only use data available at prediction time

**Data leakage** is the #1 reason models perform well in training but fail in production.

## FEATURESTORE LIFECYCLE

- **Register:** Define features with their computation logic and timestamp
- **Store:** Versioned, timestamped feature values
- **Retrieve:** Get features as-of a specific point in time
- **Lineage:** Track where each feature came from

```
from kailash_ml import FeatureStore

store = FeatureStore()
store.register_feature(
    name="avg_monthly_revenue",
    entity="user_id",
    timestamp_col="as_of_date"
)
# Retrieve point-in-time correct features
features = store.get_features(
    entity_ids=user_ids,
    as_of="2024-01-01"
)
```

## CAPSTONE ASSESSMENT CRITERIA

| Criterion                    | What We Look For                                          | Weight |
|------------------------------|-----------------------------------------------------------|--------|
| <b>Pipeline completeness</b> | All 7 steps executed (load to report)                     | 20%    |
| <b>Feature engineering</b>   | Features have domain rationale, not arbitrary             | 15%    |
| <b>Statistical rigour</b>    | Correct hypothesis formulation, proper CI interpretation  | 20%    |
| <b>Model interpretation</b>  | Coefficients, significance, model fit explained correctly | 20%    |
| <b>Communication</b>         | Report accessible to non-technical reader                 | 15%    |
| <b>FeatureStore usage</b>    | Features stored with point-in-time correctness            | 10%    |

**Common deductions:** CI interpreted as "probability" (Lesson 2.2). p-value interpreted as  $P(H_0)$  (Lesson 2.3). Accuracy reported without confusion matrix on imbalanced data (Lesson 2.6).

## CAPSTONE WORKFLOW: PUTTING IT ALL TOGETHER

---

```
from kailash_ml import (
    DataExplorer, FeatureEngineer, FeatureStore,
    TrainingPipeline, ModelVisualizer, ExperimentTracker
)
import polars as pl

# Step 1: Load
df = pl.read_csv("data/wine_quality.csv")

# Step 2: Describe
explorer = DataExplorer()
profile = explorer.profile(df)

# Step 3-4: Hypothesise and Test
tracker = ExperimentTracker()
tracker.create_experiment(name="wine_quality_analysis",
    hypothesis="Alcohol content positively affects quality")

# Step 5: Engineer and Store Features
fe = FeatureEngineer()
df = fe.add_interaction(df, col_a="alcohol", col_b="volatile_acidity")
store = FeatureStore()
store.store_features(df, entity="wine_id", timestamp="analysis_date")

# Step 6: Model
pipeline = TrainingPipeline()
pipeline.fit(df, target="quality", model="linear_regression")

# Step 7: Interpret and Report
print(pipeline.summary())
viz = ModelVisualizer()
viz.residual_plot(pipeline)
viz.coefficient_plot(pipeline)
```

# KAILASH BRIDGE: THE COMPLETE M2 ENGINE MAP

## LESSON → ENGINE MAPPING

| Lesson                  | Kailash Engine                     |
|-------------------------|------------------------------------|
| 2.1 Probability         | ExperimentTracker (Bayesian)       |
| 2.2 Estimation          | TrainingPipeline (MLE)             |
| 2.3 Bootstrap           | ExperimentTracker (resampling)     |
| 2.4 A/B Testing         | ExperimentTracker (lifecycle)      |
| 2.5 Linear Regression   | TrainingPipeline + ModelVisualizer |
| 2.6 Logistic Regression | TrainingPipeline + ModelVisualizer |
| 2.7 CUPED / DiD         | ExperimentTracker (CUPED)          |
| 2.8 Capstone            | FeatureEngineer + FeatureStore     |

**M3 Preview:** In Module 3 you take these statistical tools and build a full ML pipeline: preprocessing, model selection, hyperparameter tuning, and evaluation. The regression models from M2 become one option among many.

### Key engines in M3:

- TrainingPipeline (expanded)
- HyperparameterSearch
- ModelRegistry
- PreprocessingPipeline

## FORMULA REFERENCE: PROBABILITY (2.1)

---

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Bayes' Theorem

$$P(A, B) = P(A) \times P(B|A)$$

Joint Probability (Product Rule)

$$E[X] = \sum_i p_i x_i$$

Expected Value

$$P(B) = P(B|A)P(A) + P(B|A')P(A')$$

Total Probability

## FORMULA REFERENCE: ESTIMATION (2.2)

---

$$\sigma^2 = \frac{1}{N} \sum (x_i - \mu)^2$$

Population Variance

$$\ell(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Log-Likelihood

$$s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$$

Sample Variance (Bessel's Correction)

$$\bar{x} \pm z_{\alpha/2} \cdot \frac{s}{n}$$

Confidence Interval

## FORMULA REFERENCE: BOOTSTRAP AND TESTING (2.3)

---

$$CI_{1-\alpha} = \left[ \hat{\theta}_{\alpha/2}^*, \hat{\theta}_{1-\alpha/2}^* \right]$$

Bootstrap Percentile CI

$$\alpha_{\text{adj}} = \frac{\alpha}{m}$$

Bonferroni Correction

$$T = \frac{\bar{x} - \mu_0}{s / \sqrt{n}}$$

Test Statistic

$$n = \frac{(z_{\alpha/2} + z_{\beta})^2 \cdot 2\sigma^2}{\delta^2}$$

Minimum Sample Size (Power Analysis)

## FORMULA REFERENCE: SRM DETECTION (2.4)

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

Chi-Squared Test for Sample Ratio Mismatch

- $O_i$  = observed count in group  $i$
- $E_i$  = expected count in group  $i$
- Under  $H_0$ : groups have the expected ratio (e.g., 50/50)
- $df$  = number of groups – 1

**SRM is a safety check.** Run it before analysing any A/B test. If SRM is detected ( $p < 0.01$ ), stop and investigate. Your randomisation is broken.

## FORMULA REFERENCE: REGRESSION (2.5)

---

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

OLS Normal Equations

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

R-Squared (Coefficient of Determination)

$$t = \frac{\hat{\beta}}{SE(\hat{\beta})}$$

T-Statistic for Coefficient Significance

$$F = \frac{SS_{\text{reg}}/k}{SS_{\text{res}}/(n - k - 1)}$$

F-Statistic (Model Significance)

## FORMULA REFERENCE: LOGISTIC REGRESSION AND ANOVA (2.6)

---

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid Function

$$\text{Odds Ratio} = e^{\beta_1}$$

Odds Ratio Interpretation

$$\log \frac{P}{1 - P} = \beta_0 + \beta_1 x_1 + \dots$$

Log-Odds (Logit Link)

$$F = \frac{MS_{\text{between}}}{MS_{\text{within}}}$$

ANOVA F-Ratio

## FORMULA REFERENCE: CUPED AND DID (2.7)

---

$$\text{Var}(Y_{\text{adj}}) = \text{Var}(Y)(1 - \rho^2)$$

CUPED Variance Reduction

$$Y_{\text{adj}} = Y - \theta \cdot (X_{\text{pre}} - E[X_{\text{pre}}])$$

CUPED Adjusted Metric

$$\theta = \frac{\text{Cov}(Y, X_{\text{pre}})}{\text{Var}(X_{\text{pre}})}$$

CUPED Optimal Theta

$$\text{ATT} = (\bar{Y}_{t,\text{post}} - \bar{Y}_{t,\text{pre}}) - (\bar{Y}_{c,\text{post}} - \bar{Y}_{c,\text{pre}})$$

Difference-in-Differences

## PDF AND CDF: PROBABILITY IN PICTURES

### PROBABILITY DENSITY FUNCTION (PDF)

- A histogram where the total area under the curve equals 1
- The height at a point is the **density**, not the probability
- $P(a \leq X \leq b) = \int_a^b f(x) dx$  — the area under the curve between  $a$  and  $b$

### CUMULATIVE DISTRIBUTION FUNCTION (CDF)

- $F(x) = P(X \leq x)$  — the probability that the variable is at most  $x$
- Monotonically increasing from 0 to 1
- The CDF at  $x$  is the area under the PDF from  $-\infty$  to  $x$

**Practical use:** CDF tells you percentiles directly. "90th percentile" means  $F(x) = 0.90$ .

# LAW OF LARGE NUMBERS: WHY AVERAGES CONVERGE

## THE PRINCIPLE

- As you collect more data, the sample mean gets closer to the population mean
- Flip a coin 10 times: might get 70% heads
- Flip 10,000 times: will be very close to 50%
- **LLN guarantees convergence** for any distribution with a finite mean

**Casino math:** Casinos do not need to win every hand. LLN guarantees that over millions of hands, the house edge converges to the expected value. The longer you play, the more certain the casino's profit.

**Gambler's fallacy:** LLN does NOT mean that a streak of heads makes tails "due." Each flip is independent. LLN is about long-run averages, not individual outcomes.

## HYPOTHESIS TESTING: STEP-BY-STEP WORKFLOW

1. **State the question** in business terms ("Does the new layout increase revenue?")
2. **Formulate hypotheses:**  $H_0$ : no effect,  $H_1$ : there is an effect
3. **Choose**  $\alpha$  (usually 0.05)
4. **Collect data** (with power analysis done beforehand)
5. **Compute test statistic** (T, chi-squared, or permutation)
6. **Compare to critical value** or compute p-value
7. **Decide:** reject or fail to reject  $H_0$
8. **Report effect size**, not just significance

### Type I vs Type II Errors

|                | $H_0$ True          | $H_0$ False         |
|----------------|---------------------|---------------------|
| Reject         | Type I ( $\alpha$ ) | Correct             |
| Fail to Reject | Correct             | Type II ( $\beta$ ) |

Power =  $1 - \beta$  = probability of correctly detecting a real effect.

## POWER ANALYSIS: HOW MANY CUSTOMERS?

### HAWKER CENTRE A/B TEST

- **MDE ( $\delta$ ):** 5% drop in revenue (\$0.50 on \$10 avg)
- $\sigma$ : \$4.00 (estimated from historical data)
- $\alpha$ : 0.05
- **Power ( $1 - \beta$ ):** 0.80  $\rightarrow z_\beta = 0.84$

$$n = \frac{(1.96 + 0.84)^2 \times 2 \times 16}{0.25} = \frac{7.84 \times 32}{0.25} = 1,003$$

~1,003 customers per group, ~2,006 total

**At 100 customers/day:** you need ~20 days of data. Running for only 1 week (700 customers total) would be underpowered.

## READING A REGRESSION SUMMARY TABLE

| Feature             | Coefficient | Std Error | t-stat | p-value | Sig |
|---------------------|-------------|-----------|--------|---------|-----|
| (Intercept)         | 120,000     | 15,000    | 8.00   | <0.001  | *** |
| floor_area_sqm      | 5,200       | 420       | 12.38  | <0.001  | *** |
| remaining_lease_yrs | 3,800       | 680       | 5.59   | <0.001  | *** |
| storey_range_mid    | 2,100       | 310       | 6.77   | <0.001  | *** |
| dist_to_mrt_km      | -18,500     | 4,200     | -4.40  | <0.001  | *** |
| is_mature_estate    | 42,000      | 8,500     | 4.94   | <0.001  | *** |

**Reading this table:** Each sqm adds \$5,200 to the price. Each additional km from MRT *reduces* price by \$18,500. Mature estates command a \$42,000 premium. All coefficients are highly significant ( $p < 0.001$ ).  $R^2 = 0.82$ ,  $F = 450$  ( $p < 0.001$ ).

## RESIDUAL ANALYSIS: IS YOUR MODEL VALID?

### OLS ASSUMPTIONS (CHECK WITH RESIDUALS)

- **Linearity:** Residuals vs fitted plot should show no pattern
- **Normality:** Residuals should be approximately Normal (Q-Q plot)
- **Homoscedasticity:** Residual variance should be constant (no funnel shape)
- **Independence:** Residuals should not be autocorrelated

**Heteroscedasticity:** If the residual spread increases with the predicted value (funnel shape), your standard errors are wrong and t-statistics are unreliable. Fix: use robust standard errors or transform the target ( $\log y$ ).

**ModelVisualizer automates this:**  
`viz.residual_plot(pipeline)` creates residuals vs fitted, Q-Q, and scale-location plots in one call.

## ODDS VS PROBABILITY: A VISUAL COMPARISON

| Probability | Odds | Description                         |
|-------------|------|-------------------------------------|
| 0.50        | 1:1  | Even chance                         |
| 0.75        | 3:1  | Three times more likely yes than no |
| 0.80        | 4:1  | Four times more likely              |
| 0.90        | 9:1  | Nine times more likely              |
| 0.10        | 1:9  | Nine times more likely no than yes  |

### Conversion formulas:

- $\text{Odds} = P / (1 - P)$
- $P = \text{Odds} / (1 + \text{Odds})$

**Common confusion:** "4x the odds" does NOT mean "4x the probability." If baseline  $P = 10\%$  (odds = 0.11), 4x odds = 0.44, so  $P = 31\%$ . The probability increased 3.1x, not 4x.

## CROSS-ENTROPY: THE CLASSIFICATION LOSS FUNCTION

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Binary Cross-Entropy Loss (negated log-likelihood)

- When  $y_i = 1$ : loss =  $-\log \hat{p}_i$  (penalise low predicted probability)
- When  $y_i = 0$ : loss =  $-\log(1 - \hat{p}_i)$  (penalise high predicted probability)
- Perfect prediction ( $\hat{p} = y$ ) gives loss = 0

### Why cross-entropy, not squared error?

For binary outcomes, squared error has multiple local minima. Cross-entropy is convex in the parameters, guaranteeing a unique optimum. It also connects to information theory: cross-entropy measures how different the predicted distribution is from the true distribution.

**Deep learning bridge:** This exact loss function is used for training neural network classifiers. You will see it again in M4 (Deep Learning).

## CUPED: STEP-BY-STEP IMPLEMENTATION

1. **Identify pre-experiment covariate** ( $X_{\text{pre}}$ ):  
same metric from the period before the experiment
2. **Compute  $\theta$** :  $\theta = \text{Cov}(Y, X_{\text{pre}}) / \text{Var}(X_{\text{pre}})$
3. **Adjust the metric**:  $Y_{\text{adj}} = Y - \theta \cdot (X_{\text{pre}} - \bar{X}_{\text{pre}})$
4. **Run your test** on  $Y_{\text{adj}}$  instead of  $Y$
5. **Quantify improvement**: compare  $\text{Var}(Y_{\text{adj}})$  to  $\text{Var}(Y)$

```
import polars as pl
import numpy as np

# Compute theta (regression coefficient)
cov = df.select(
    pl.corr("revenue_post", "revenue_pre")
    * pl.col("revenue_post").std()
    * pl.col("revenue_pre").std()
).item()
var_pre = df["revenue_pre"].var()
theta = cov / var_pre

# Adjust metric
mean_pre = df["revenue_pre"].mean()
df = df.with_columns(
    (pl.col("revenue_post")
     - theta * (pl.col("revenue_pre") - mean_pre)
    ).alias("revenue_adj")
)

# Variance reduction
print(f"Original var: {df['revenue_post'].var():.2f}")
print(f"Adjusted var: {df['revenue_adj'].var():.2f}")
```

# FORMULATING EXPERIMENT HYPOTHESES

---

## GOOD HYPOTHESES

- **Specific:** "BOGO promotion gives higher average spend than 20% discount"
- **Measurable:** defines a metric (average spend)
- **Testable:** can design an experiment to test it
- **Falsifiable:** possible to observe evidence against it

## BAD HYPOTHESES

- "The new design is better" — better how? No metric.
- "Users prefer our product" — not testable with A/B.
- "Revenue will increase eventually" — not falsifiable.

**Template:** "Changing [feature] from [A] to [B] will [increase/decrease] [metric] by [amount] within [timeframe]."

## PARAMETRIC VS NON-PARAMETRIC BOOTSTRAP

### NON-PARAMETRIC BOOTSTRAP

- Resample directly from the data
- No assumptions about the distribution
- Works for any statistic
- Requires sufficient sample size ( $n \geq 30$ )

**Use when:** you do not know the distribution, or the statistic has no closed-form standard error.

### PARAMETRIC BOOTSTRAP

- Assume a distribution (e.g., Normal)
- Estimate parameters from data
- Generate new samples from the fitted distribution
- More powerful when the assumption is correct

**Use when:** you are confident in the distributional assumption AND the non-parametric bootstrap gives unstable results (very small  $n$ ).

## TOTAL PROBABILITY: THE HAWKER CENTRE QUEUE

### SCENARIO

- A food court has 3 stalls: A (40% of customers), B (35%), C (25%)
- Wait time exceeds 10 min: Stall A: 20%, Stall B: 30%, Stall C: 10%
- **What is the overall probability of waiting > 10 min?**

$$\begin{aligned} P(\text{Wait}) &= \sum_i P(\text{Wait}|\text{Stall}_i) \cdot P(\text{Stall}_i) \\ &= 0.20 \times 0.40 + 0.30 \times 0.35 + 0.10 \times 0.25 \\ &= 0.08 + 0.105 + 0.025 = 0.21 \end{aligned}$$

**Total probability** computes the overall probability by summing over all possible pathways, each weighted by its probability of occurring.

## CROSS-VALIDATION: PREVENTING OVERFITTING

---

### TRAIN/TEST SPLIT

- Fit the model on 80% of the data (training set)
- Evaluate on the remaining 20% (test set)
- If training R-squared is 0.95 but test R-squared is 0.60, the model is **overfitting**

### K-FOLD CROSS-VALIDATION

- Split data into  $k$  equal folds (e.g.,  $k = 5$ )
- Train on  $k - 1$  folds, test on the remaining fold
- Repeat  $k$  times, average the results
- More reliable than a single train/test split

**Detailed in M3.2.** For now, know that cross-validation prevents you from being fooled by a model that memorises the training data.

## SRM: COMMON CAUSES AND HOW TO FIX THEM

| Cause                       | Symptom                                        | Fix                                            |
|-----------------------------|------------------------------------------------|------------------------------------------------|
| <b>Bot filtering</b>        | One variant loses more users to bot detection  | Apply bot filter BEFORE assignment             |
| <b>Redirect bugs</b>        | Treatment variant has higher page load failure | Fix the redirect; verify with monitoring       |
| <b>Population filtering</b> | Treatment only shown to logged-in users        | Apply same eligibility criteria to both groups |
| <b>Caching</b>              | CDN serves wrong variant to some users         | Cache-bust per-user assignment                 |

### If SRM is detected:

1. Stop the experiment analysis
2. Investigate the cause
3. Fix the issue
4. Re-run the experiment

Do NOT proceed with analysis on SRM-contaminated data. The treatment and control are no longer comparable.

## MAP ESTIMATION AND THE REGULARISATION CONNECTION

### MAP = MLE + PRIOR

$$\hat{\theta}_{MAP} = \arg \max_{\theta} [\ell(\theta) + \log P(\theta)]$$

Log-likelihood plus log-prior

- **Gaussian prior** on  $\theta$ :  $P(\theta) \propto e^{-\lambda \|\theta\|_2^2}$
- $\log P(\theta) = -\lambda \|\theta\|_2^2 + C$
- This is **L2 regularisation** (Ridge regression)

### LAPLACE PRIOR = L1 (LASSO)

- **Laplace prior**:  $P(\theta) \propto e^{-\lambda |\theta|}$
- Produces **sparse solutions** (some coefficients exactly zero)
- This is **L1 regularisation** (Lasso regression)

**Bridge to M3:** Ridge and Lasso are covered in M3 as regularised ML models. Now you know their Bayesian origin: they are MAP estimation with specific priors.

## PRESENTING STATISTICAL FINDINGS

### DO

- **Lead with the business answer:** "Yes, the price increase reduced revenue by 5%"
- **Quantify uncertainty:** "We are 95% confident the true reduction is between 2% and 8%"
- **Visualise key results:** One chart is worth a thousand p-values
- **State limitations:** What the analysis cannot tell you
- **Recommend actions:** Based on the evidence

### DO NOT

- Lead with p-values or test statistics
- Show raw regression output tables without explanation
- Claim causation without experimental design
- Report only "significant" results (publication bias)
- Use jargon: "We reject the null at the 5% level"

**The capstone report is graded on communication (15%).** Technical correctness without clarity is incomplete.

## POINT-IN-TIME CORRECTNESS: THE SUBTLE DATA LEAKAGE

### SCENARIO

- You build a model in March 2025 to predict January 2025 sales
- Your feature "average\_monthly\_spend" is computed from all of 2024
- **This is correct:** December 2024 data was available in January 2025

### BUT WHAT IF...

- Your feature includes a "total\_2024\_spend" that was only finalised in February 2025
- In January 2025, the system only had 11 months of data
- **Leakage:** your model uses future information

**FeatureStore enforces this:** When you request features "as of January 1, 2025", the store only returns feature values that were computed and available on that date. No future data leaks in.

**Real-world consequence:** A model with data leakage shows  $R^2 = 0.95$  in backtesting but  $R^2 = 0.40$  in production. The gap is the leaked signal.

# CHOOSING YOUR STATISTICAL TOOL

## DECISION GUIDE

- **Outcome is continuous?**
  - 2 groups → T-test
  - 3+ groups → ANOVA
  - With predictors → Linear Regression
- **Outcome is binary (yes/no)?**
  - → Logistic Regression
- **Comparing proportions?**
  - → Chi-squared test
- **No distributional assumptions?**
  - → Permutation test or Bootstrap

### M2 Toolkit Summary:

| Tool                | Lesson |
|---------------------|--------|
| Bayes' Theorem      | 2.1    |
| MLE / MAP           | 2.2    |
| Bootstrap CI        | 2.3    |
| A/B Test Design     | 2.4    |
| Linear Regression   | 2.5    |
| Logistic Regression | 2.6    |
| CUPED / DiD         | 2.7    |
| Feature Engineering | 2.8    |

# CLEAN DATAOPS: FROM SILOS TO PIPELINES

---

## THE PROBLEM

- Marketing tracks clicks in Google Analytics
- Sales tracks revenue in Salesforce
- Product tracks usage in Mixpanel
- Finance tracks costs in SAP
- **Nobody can join these datasets** without manual export/import

## CLEAN ARCHITECTURE

- **Data schema:** agreed field names, types, granularity
- **Data transfer:** automated ETL pipelines (not CSV email)
- **Processing logic:** version-controlled transformations
- **Connectors:** standard APIs to each source

**Kailash DataFlow** provides connectors, schema validation, and automated transfers. Data collection plans from Lesson 2.4 map directly to DataFlow pipelines.

## MODULE 2: KEY TAKEAWAYS

---

### STATISTICAL FOUNDATIONS

- **Probability** is the language of uncertainty. Bayes' theorem updates beliefs with evidence.
- **Estimation** connects data to parameters. MLE maximises likelihood; MAP adds priors.
- **Hypothesis testing** is decision-making under uncertainty. p-value is NOT  $P(H_0)$ .
- **Bootstrap** works for any statistic without distributional assumptions.

### MODELLING AND EXPERIMENTS

- **Linear regression** quantifies relationships.  $R^2$ ,  $t$ -stat,  $F$ -stat evaluate fit and significance.
- **Logistic regression** predicts binary outcomes. Odds ratios interpret coefficients.
- **CUPED** reduces experiment variance using pre-experiment data.
- **Feature engineering** creates better model inputs. FeatureStore prevents data leakage.

## M2 COMMON MISTAKES CHECKLIST

---

| # | Mistake                                 | Correction                                                       |
|---|-----------------------------------------|------------------------------------------------------------------|
| 1 | CI = "95% chance parameter is in range" | 95% of intervals from repeated experiments contain the parameter |
| 2 | p-value = $P(H_0 \text{ is true})$      | p-value = $P(\text{data this extreme} \mid H_0)$                 |
| 3 | "Accept $H_0$ "                         | "Fail to reject $H_0$ "                                          |
| 4 | Odds ratio = risk ratio                 | Odds ratio approximates risk ratio only for rare events          |
| 5 | High $R^2$ = good model                 | Check residuals, use test set, consider adjusted $R^2$           |
| 6 | Accuracy on imbalanced data             | Use confusion matrix, precision, recall                          |
| 7 | SRM ignored                             | Always check SRM before analysing A/B results                    |
| 8 | No power analysis                       | Compute sample size before running experiment                    |

## WHAT COMES NEXT: MODULE 3 PREVIEW

---

### M3: SUPERVISED ML — THEORY TO PRODUCTION

- **3.1** Feature Engineering (deep dive with FeatureEngineer)
- **3.2** Cross-Validation and Model Selection
- **3.3** Decision Trees and Random Forests
- **3.4** Gradient Boosting (XGBoost, LightGBM)
- **3.5** Classification Metrics and Evaluation
- **3.6** Hyperparameter Tuning
- **3.7** Model Registry and Deployment
- **3.8** Capstone: Full ML Pipeline

#### How M2 Prepares You for M3:

- Regression (2.5) → becomes one model option among many
- Logistic regression (2.6) → baseline classifier
- Cross-validation (2.5 preview) → full treatment in 3.2
- Feature engineering (2.8) → automated with FeatureEngineer
- MLE (2.2) → loss functions for all models
- Bootstrap (2.3) → confidence intervals for model metrics

## END OF MODULE ASSESSMENT

---

### QUIZ (INDIVIDUAL)

- Covers all 8 lessons
- Mix of conceptual and computational questions
- Expect: Bayes' theorem calculation, CI interpretation, p-value interpretation, regression output reading, odds ratio computation
- No code required, but understanding of Kailash engine outputs

### CAPSTONE PRESENTATION (TEAM)

- 5-minute presentation of capstone findings
- Audience: non-technical stakeholders
- Must include: business question, methodology summary, key findings, recommendations, limitations
- Rubric: technical correctness (40%), communication clarity (30%), code quality (20%), FeatureStore usage (10%)

# MODULE 2 COMPLETE

## STATISTICAL MASTERY FOR MACHINE LEARNING AND AI SUCCESS

You can now reason about uncertainty, test hypotheses, build regression models, and design experiments with statistical rigour.

*"The goal is not to compute a  $p$ -value. The goal is to make a better decision."*

MLFP — ML Foundations for Professionals — Terrene Open Academy

Powered by Kailash Python SDK — Terrene Foundation